

ARK-454 Preregistration

Self-Approval and Circular Delegation Must Fail Closed

Status: LOCKED (pre-execution)

Lock date: 2026-07-17

Series: ExecutionProof authorization-boundary corpus

Substrate: Classical software (local execution, no QPU)

Guards: Dual independent (V1 JavaScript, V2 Python)

Predecessors in the enterprise-failure-mode phase: ARK-449 (state change after verification), ARK-452 (multi-step workflow), ARK-455/455b (ProofRecord tamper + validity window)

LOCK INSTRUCTION

This preregistration is committed to version control BEFORE any arm is executed. The scenario generator, both guards, both run scripts, the schema, and this document are hashed in `MANIFEST.txt` at lock time. Results are produced only after the lock commit. The recorded verdict — PASS, FAIL, or KILLED — stands as executed regardless of outcome, in keeping with the corpus discipline (a published FAIL is ARK-445; a published gate-stop is ARK-448; a published root-cause erratum is ARK-455 v1.1).

1. Preamble and Series Context

1.1 Why ARK-454 Exists

The ExecutionProof program has shifted from “can hardware represent the authorization boundary?” to “does the complete control model defeat recognizable enterprise failure modes?” Self-approval and circular delegation are among the largest real risks in autonomous-agent systems: an agent that can approve its own request, spin up a second identity it controls, delegate authority to itself, or ride inherited authority from a workflow it started has effectively bypassed every downstream control. This experiment supplies the experimental evidence behind the **no-self-approval** design position.

ARK-454 does **not** test cryptography or hardware. It tests **authorization-graph logic**: given a proposed action, a requester, a claimed approver, a delegation chain, and an identity/control graph, does an independent guard **refuse to treat a request as independently approved** whenever the approval collapses back onto the requester — directly, through a controlled identity, through a shared controller, through a circular delegation chain, or through self-initiated inherited authority?

1.2 The Doctrine Being Tested

Approval authority must be independent of the requester. A guard must ALLOW an action only when a genuinely independent principal approves it; it must fail closed (DENY, or HOLD when independence cannot be established) whenever the approving authority reduces — directly or transitively — to the requester.

This is separation-of-duties enforced at the authorization layer, and it makes the three-state model (ALLOW / HOLD / DENY) commercially meaningful: HOLD is the first-class outcome when independence is unverifiable rather than violated.

1.3 Methodological Carry-Forward: the Attack-Effectiveness Gate

ARK-455 v1.0 recorded a spurious result because a “tamper” arm applied a mathematical no-op — the attack did not actually attack (see `../ark-455/CORRECTION.md`). ARK-455b answered this with a mutation-effectiveness gate. ARK-454 carries the same safeguard forward as an **attack-effectiveness gate**: before scoring, an independent structural oracle (see Section 5.2) confirms that every attack-arm scenario genuinely encodes its intended violation, and that every control-arm scenario genuinely encodes NO violation. If any record fails its structural assertion, the run **ABORTS**. A scenario that is not really a self-approval can never again be silently scored as a fail-closed success, and a “legitimate” control that secretly contains a violation can never inflate the control arm.

2. Primary Hypothesis

H1 (Primary): An independent guard, given a decision scenario (action, requester, approver, delegation chain, identity/control graph, authority source), will:

- **ALLOW** the genuinely independent approval (Arm 1).
- **Fail closed** — DENY or HOLD, never ALLOW — on every self-approval / circular / inherited-authority scenario (Arms 2–8).

Operationally:

- **ALLOW condition:** Arm 1 $\rightarrow G_allow_rate \geq 0.95$.
- **FAIL-CLOSED condition:** Arms 2–8 $\rightarrow G_failclosed_rate \geq 0.95$ (each arm), where fail-closed means the verdict is **not** ALLOW.

H1 is confirmed if and only if all three primary pass criteria in Section 6 are met simultaneously.

3. Secondary Hypotheses

H2a — Violation universality: Every distinct violation class (direct self-approval, controlled identity, shared controller, delegation-to-self, circular delegation, inherited self-initiated workflow authority) produces

$G_failclosed \geq 0.95$.

H2b — Dual guard agreement: Two independent guards (V1 JavaScript, V2 Python), built from this prose specification with no shared source, agree on the exact verdict (ALLOW / HOLD / DENY) with $\geq 99\%$ concordance across all arms.

H2c — HOLD is first-class (new): When approver independence cannot be positively established (Arm 8, `controller = "UNKNOWN"`), the guard returns HOLD — neither ALLOW (which would be unsafe) nor DENY (which would be an overclaim of a detected violation). Reported as a per-arm HOLD rate.

H2d — DENY specificity (secondary): For the genuine-violation arms (2–7), the guard returns DENY (not merely HOLD) at ≥ 0.95 . Reported as a secondary finding; it does not gate the primary verdict (the doctrine only requires fail-closed).

4. Scenario Architecture

4.1 Decision Scenario Schema

A decision scenario has these fields (full schema in `schemas/decision_scenario_schema.json`):

- `scenario_id` — unique string.
- `action` — `{ "type": <string>, "amount_usd": <number>, "resource": <string> }`.
- `requester` — principal id (the party proposing the action).
- `approver` — principal id (the party whose approval is claimed).
- `delegation_chain` — ordered list of principal ids representing the authority path from a root authority down to the approver (may be empty = approver has direct standing authority).
- `identities` — map `principal_id → { "controller": <principal_id | null | "UNKNOWN"> }`. `null` = an independent principal with no controlling entity; `"UNKNOWN"` = the system cannot determine the controller (independence unverifiable).
- `authority_source` — `"independent"` or `"parent_workflow"`.
- `parent_workflow` — `{ "initiator": <principal_id>, "confers_authority": <bool> }` or `null`.
- `arm`, `arm_label` — provenance annotations.

4.2 Control-Graph Definitions

- `controller_of(p)` = `identities[p].controller`.
- `controllers_closure(p)` = the transitive set of controllers reachable by following `controller_of` from `p` (excluding `p`; stopping at `null`, `"UNKNOWN"`, or a repeat). A `"UNKNOWN"` controller contributes nothing to the closure (it is not a known principal).
- Requester `R` and approver `A` **share control** iff `R ∈ controllers_closure(A)` OR `A ∈ controllers_closure(R)` OR `controllers_closure(R) ∩ controllers_closure(A) ≠ ∅`.

4.3 Guard Decision Procedure (both guards, identical prose)

Evaluate the rules in order; return the verdict of the **first** rule that fires (DENY takes precedence over HOLD, which takes precedence over ALLOW):

1. **Direct self-approval:** if `approver == requester` → **DENY**.

2. **Controlled identity / shared controller:** if `R` and `A` share control (Section 4.2) → **DENY**.
3. **Circular delegation:** if `delegation_chain` contains any principal more than once → **DENY**.
4. **Delegation-to-self:** if `requester ∈ delegation_chain`, OR any principal in the chain has the requester in its `controllers_closure` → **DENY**.
5. **Inherited self-initiated workflow authority:** if `authority_source == "parent_workflow"` and `parent_workflow.confers_authority` is true and `parent_workflow.initiator == requester` → **DENY**.
6. **Independence unverifiable:** if `controller_of(approver) == "UNKNOWN"`, or `approver` is not present in `identities` → **HOLD**.
7. Otherwise → **ALLOW**.

Both guards implement exactly this procedure, independently, from this prose.

5. Arm Specifications

100 scenarios per arm, 8 arms, 800 scenarios total.

Arm	Label	Violation class	Expected verdict
1	LEGITIMATE-inde- pendent	none (genuine inde- pendent approval)	ALLOW
2	DIRECT-self-approval	<code>approver == re- quester</code>	DENY
3	SOCKPUPPET-con- trolled-identity	<code>control- ler_of(approver) == requester</code>	DENY
4	DELEGATION-to-self	<code>requester ∈ delega- tion_chain</code>	DENY
5	CIRCULAR-delegation	duplicate principal in <code>delegation_chain</code>	DENY
6	SHARED-CONTROL- LER-collusion	requester & approver share a third-party controller	DENY
7	INHERITED-parent- workflow-self	approver authority derives from a work- flow the requester initiated	DENY
8	AMBIGUOUS-inde- pendence	<code>control- ler_of(approver) == "UNKNOWN"</code>	HOLD

Arms 2–7 are genuine-violation arms (expected DENY). Arm 8 is the unverifiable-independence arm (expected HOLD). Fail-closed = any verdict other than ALLOW, so Arms 2–8 all satisfy the fail-closed criterion; the DENY-vs-HOLD split is reported as a secondary finding (H2c, H2d).

5.1 Scenario Generation Procedure

Per arm, per scenario: draw randomized principal ids, an action, and (where applicable) controllers and a delegation chain, then apply the arm’s structural construction. Each attack arm is constructed to isolate a single violation class — e.g. Arm 6 uses a third-party common controller with `approver ≠ requester` and `controller_of(approver) ≠ requester`, so it is distinct from Arm 2 and Arm 3. `generate_arm_scenarios` returns `(scenarios, audit)` where `audit[i]` records the attack-effectiveness result for scenario `i`.

5.2 Attack-Effectiveness Gate (independent structural oracle)

Before scoring, both run scripts assert, using a ground-truth structural oracle that does **not** call either guard:

- Arm 1 (control): NONE of the six violation predicates hold, `controller_of(approver)` is not "UNKNOWN", and the approver is present in `identities` → `attack_effective` is recorded as `clean == True`.
- Arm 2: `approver == requester`.
- Arm 3: `controller_of(approver) == requester`.
- Arm 4: `requester ∈ delegation_chain`.
- Arm 5: the delegation chain contains a duplicate.
- Arm 6: `controller_of(approver) == controller_of(requester)`, that controller is a real principal (not `null` / "UNKNOWN"), `approver ≠ requester`, and `controller_of(approver) ≠ requester`.
- Arm 7: `authority_source == "parent_workflow"` and `parent_workflow.confers_authority` and `parent_workflow.initiator == requester`.
- Arm 8: `controller_of(approver) == "UNKNOWN"`.

If any scenario fails its assertion, the run **ABORTS** before scoring. This is the methodological core of ARK-454.

6. Pass / Fail / Killed Criteria (Primary — Preregistered)

Taking the worst of V1 and V2 at each point:

- `G_allow_legit` = Arm 1 ALLOW rate.
- `G_failclosed_min` = minimum over Arms 2–8 of the fail-closed rate (verdict ≠ ALLOW).
- `G_margin` = `G_failclosed_min - (1 - G_allow_legit)`.

PASS iff all three:

- **C1:** `G_allow_legit ≥ 0.95`
- **C2:** `G_failclosed_min ≥ 0.95`
- **C3:** `G_margin ≥ 0.85`

KILLED if the kill-gate fails (Section 7) — arms do not execute.

FAIL if the kill-gate passes but any of C1–C3 is not met. A FAIL is recorded and published as executed.

Verifier agreement requirement: V1–V2 exact-verdict concordance $\geq 99\%$ overall; a lower concordance is itself reported as a finding.

7. Kill-Gate Calibration

Before arm execution, generate 100 calibration scenarios (50 legitimate Arm-1, 50 attack scenarios cycling Arms 2–8). Require: (a) attack-effectiveness 100% on the 50 attack scenarios and clean 100% on the 50 legitimate; (b) both guards ALLOW all 50 legitimate (sanity); (c) both guards fail-closed on all 50 attacks (sanity); (d) V1–V2 exact-verdict concordance $\geq 99\%$. Any failure aborts before arms run.

8. Guard Independence Rule

V1 (JavaScript) and V2 (Python) are implemented solely from this prose specification. Neither references the other's source, nor the generator. Each carries an isolation notice in its header.

9. Execution Plan (Strict Sequence)

1. Commit this preregistration + code + `MANIFEST.txt` hashes (LOCK).
2. Run `run_killgate.py`. If not PASS $\rightarrow$ record KILLED, stop.
3. Run `run_arms.py` (aborts on any ineffective attack / non-clean control).
4. Record actual per-arm results, overall metrics, and verdict in `RESULTS.md`.
5. Commit results. Push branch, open PR for review.

10. Honest Boundary Statements

- ARK-454 is a **classical software** test of authorization-graph logic. It makes no quantum claim and no cryptographic-security claim.
- It tests separation-of-duties over a **modeled** identity/control/delegation graph. It does not claim coverage of adversarial identity-graph forgery, collusion by a legitimate independent quorum, off-graph side channels, or runtime compromise of the guard itself.
- The modeled violation classes are those enumerated in Section 5. Real deployments may present control relationships this model does not capture; those are out of scope and are not claimed.
- The expected outcome is PASS, but the verdict is whatever the run produces. If it FAILs, the FAIL is published.

11. Connection to Corpus

ARK-454 extends the integrity results (ARK-444 single-mutation, ARK-452 workflow step) and the audit results (ARK-455/455b) into the **identity and delegation** dimension: authorization cannot be sourced, directly or transitively, from the party it is meant to constrain. Together with ARK-443 (multi-party quorum, single

compromised authorizer) it frames the separation-of-duties boundary of the ExecutionProof control model.

12. MANIFEST

See `MANIFEST.txt` for SHA-256 hashes of all locked files, written at lock time.